

Conditional Expressions & Operators

In this section of the course we are going to be focusing on being able to conduct logical operations and functionality within our SQL code. We will learning about keywords:

- CASE
- COALESCE
- NULLIF
- CAST

We are also going to learn about some additional functionality which allows us to expand on the usefulness of pgAdmin:

- Views
- Import and Export Functionality

These keywords and functions will allow us to add logic to our commands and workflows in SQL.

CASE

The **CASE** statement is used to only execute SQL code when certain conditions are met. This is very similar to **IF/ELSE** statements in other programming languages.

There are 2 ways to use a CASE statement, either a general CASE or a CASE expression. Both methods can lead to the same results.

- General Syntax:

```
CASE  
  WHEN condition1 THEN result1  
  WHEN condition2 THEN result2  
  ELSE some_other_result  
END;
```

Simple Example:

```
SELECT * FROM test;
```

a
1
2

```
SELECT a,  
       CASE  
         WHEN a = 1 THEN 'one'  
         WHEN a = 2 THEN 'two'  
         ELSE 'other' AS label  
END  
FROM test;
```

a	label
1	one
2	two

The **CASE** expression syntax first evaluates an expression then compares the result with each value in the **WHEN** clauses sequentially.

- Expression Syntax

```
CASE expression  
  WHEN condition1 THEN result1  
  WHEN condition2 THEN result2  
  ELSE some_other_result  
END;
```

Simple Example:

```
SELECT a,  
       CASE a  
         WHEN 1 THEN 'one'  
         WHEN 2 THEN 'two'  
         ELSE 'other'  
       END AS label  
FROM test;
```

a	label
1	one
2	two

COALESCE Function

The **COALESCE** function accepts an unlimited amount of arguments. It returns the first argument that is not null. If all arguments are null, the **COALESCE** function will return null.

Example:

- `SELECT COALESCE(1, 2)`
 - 1
- `SELECT COALESCE(NULL, 2, 3)`
 - 2

This function is typically used when querying a table that contains null values and substituting it with another value.

Example:

- Table of products (Price & Discount in Dollars)

Item	Price	Discount
A	100	20
B	300	null
C	200	10

The **COALESCE** can be used to replace the null values with 0s for a mathematical calculation without having to affect or alter your original table.

- What is the final price?

Item	Price	Discount
A	100	20
B	300	null
C	200	10

**SELECT item,(price - discount) AS final
FROM table**

Item	final
A	80
B	null
C	190

You are able to calculate the final price of both item A and C, but since the discount for item B was null, SQL can't subtract null with an integer as they are both incompatible data types. In this scenario SQL tries to relay that one of the values during the mathematical operation was null by returning the null value.

Doesn't work for item B, should be 300.

COALESCE Correct Example:

```
SELECT item, (price - COALESCE(discount, 0))  
AS final FROM table;
```

Item	final
A	80
B	300
C	190

Keep the **COALESCE** function in mind in case you encounter a table with the null values that you want to perform operations on!

CAST

The **CAST** operator lets you convert from one data type into another. Keep in mind that not every instance of a data type can be **CAST** to another data type, it must be reasonable to convert the data, for example '5' to an integer will work, 'five' to an integer will not.

- Syntax for CAST function

```
SELECT CAST('5' AS INTEGER)
```

- PostgreSQL CAST operator

```
SELECT '5'::INTEGER
```

Keep in mind you can use this in a SELECT query with a column name instead of a single instance.

```
SELECT CAST(date AS TIMESTAMP)  
FROM table;
```

NULLIF

The NULLIF function takes in 2 inputs and returns NULL if both are equal, otherwise it returns the first argument passed.

Example:

- NULLIF(10, 10)
 - Returns NULL
- NULLIF(15, 10)
 - Returns 15

This becomes very useful in cases where a NULL value would cause an error or wanted result.

Given this table calculate the ratio of Department A to Department B

Name	Department
Lauren	A
Vinton	A
Claire	B

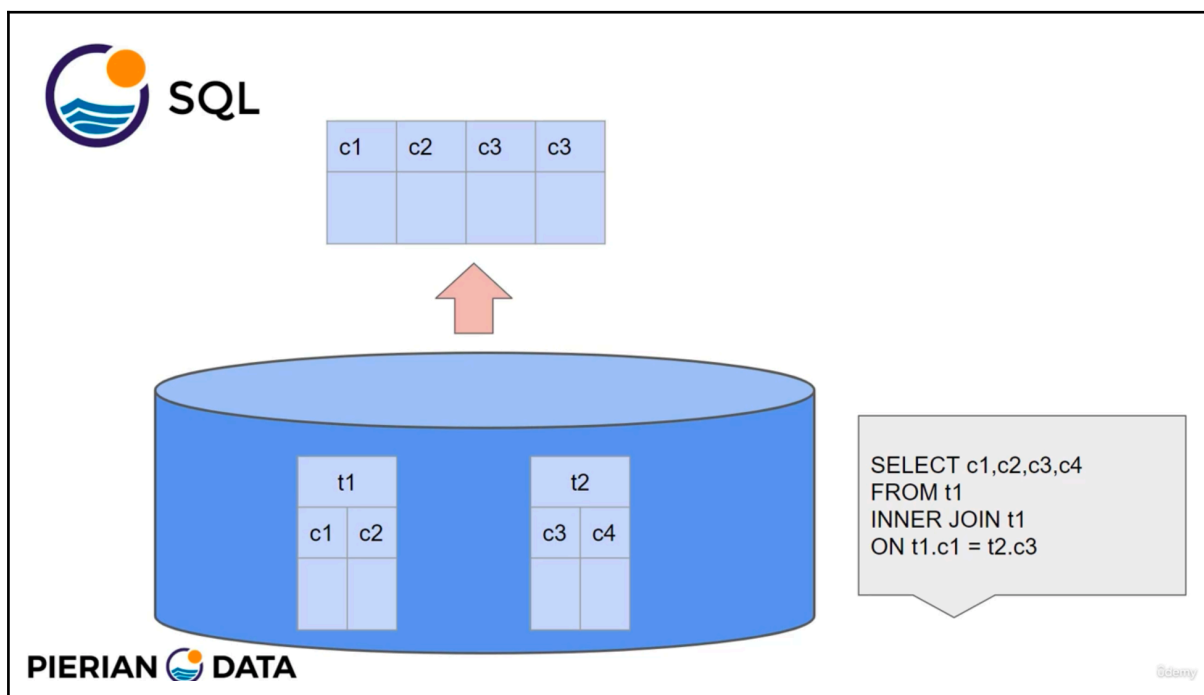
We can easily see that the ratio of A to B is 2:1 or 200%.

Let's jump to pgAdmin, quickly create this table and walk through solving the RATIO and why we may need NULLIF

VIEW

Often there are specific combinations of tables and conditions that you find. Instead of having to perform the same query over and over again as a starting point, you can create a **VIEW** to quickly see this query with a simple call.

For example, let's imagine that you have a database and you're working on a project that has something to do with the database below. As you're working on this project and writing reports, you notice that there's 2 tables that you're always joining together.



Instead of having to write that query over and over again, you can create or transform the result into a **VIEW**. This way you can instead just say **SELECT *** from the view and begin working from there.

- A view is a database object that is of a stored query.
- A view can be accessed as a virtual table in PostgreSQL.
- Notice that a view does not store data physically, it simply stores the query.

Therefore, a view simply stores the query in a way that allows you to easily execute that query as if you were selecting from an already existing table. Essentially it transforms a complex query which you are executing over and over again, just call it as if it was some already existing table from the view.

- You can also update and alter existing views.